Make Your ITEM (with number) · 2026 유니티 게임잼 기획 정리

> 숫자 함수와 영문 변수로 제작 프로그램을 작성하고, 3D 대장장이가 만든 장비를 자유롭게 팔아 공방을 성장시키는 게임.

주제 행운의 숫자 · **게임 이름** Make Your ITEM (with number) · **플랫폼** Unity 6, 3D 싱글 플레이 · **조작** 마우스 클릭과 키보드 입력

대상 Unity 게임 개발자와 코딩을 즐기는 플레이어. **제작 계획** 기능 구현에 ChatGPT, 3D 모델 제작에 Meshy 활용. 함수와 재료를 숫자로 표현하는 자체 언어를 만들고, 코딩·아이템 제작·타이쿤의 성장을 연결한다. 시스템 참고작은 《농부는 대체되었다》.

1. 현재 방향과 핵심 규칙
2. 플레이 흐름과 화면
3. 숫자 도면 언어
    - 도면 함수·표기 레퍼런스
4. 컴파일러, 가상 머신과 오류 처리
5. 판매와 발견
6. 행운의 숫자, 상점과 해금 후보
7. 게임잼 구현 우선순위
8. 보류 중인 아이디어 · 4~9번 재료와 특수 가공
    - 용뼈 정제 예시
    - 재료 조합을 발견하는 예시
9. 확정 예약어와 인게임 적용 프롬프트
    - 인게임 숫자 예약어 및 구문 강조 적용 프롬프트
10. 알고리즘·자료구조를 활용하는 추가 가공 기획
    - 기본 제작과 연결
    - 10.1. 겹겹이 감싸는 마법 주조
    - 10.2. 중복 금지 룬
    - 10.3. 마력 타격 리듬
    - 10.4. 미지의 공명 온도 찾기
    - 10.5. 통합 제작 흐름과 판매
    - 10.6. 필요한 기능과 구현 순서
    - 10.7. 후속 결정 항목
11. 오늘의 행운 함수·다이아몬드·시간 조회
    - 11.1. 확정 사용 문법
    - 11.2. 오늘의 행운 코드 생성
    - 11.3. 검 제작과 보상

11.4. 구매로 해금하는 행운 코드 조회 기능

11.5. 낮·밤과 현재 시각 조회

11.6. 날짜 변경과 저장 처리

11.7. 기존 행운 기획과의 관계

11.8. 후속 결정 항목

1. 현재 방향과 핵심 규칙

플레이어는 기본 공정을 짧게 익힌 뒤 원하는 장비를 만들고 자유 판매한다. 제작법은 재료 설명, 코드 툴팁, 공정 결과, 판매 영수증을 통해 스스로 발견한다. 현재의 기본 플레이 흐름은 자유 제작과 판매다.

> 같은 재료라도 어느 공정에 어떤 상태에서 쓰느냐에 따라 결과가 달라진다. 판매가는 완성품이 실제로 얻은 성질에 따라 올라가고, 이익은 그 가격에서 제작비를 뺀 금액이다.

예시 뜨거운 철 검날을 물로 식히면 담금질로 경도가 오른다. 식은 검날에 물을 쓰면 물값이 들고 경도 변화는 없다. 나무는 부품 장착에서 손잡이, 화로에서 연료가 될 수 있다. 플레이어는 코드의 순서, 조건, 재료를 바꾸며 어떤 변경이 수익으로 이어지는지 알아간다.

2. 플레이 흐름과 화면

1. **튜토리얼** 철을 올리고 가열·타격·냉각해 검날을 판매하는 기본 루프를 한 번 경험한다.
2. **코딩과 시험 실행** 숫자 도면을 작성하고, 재료를 소비하지 않는 시험에서 줄별 결과와 예상 비용을 확인한다.
3. **실제 제작** 컴파일한 도면을 대장장이가 실행한다. 공정마다 제작 시간이 걸리고 3D 대장간에 행동과 상태 변화가 드러난다.
4. **검사와 자유 판매** 완성품의 성질과 가격 내역을 확인한 뒤 판매한다. 완성 검 외에 판매 조건을 갖춘 검날도 팔 수 있다.
5. **상점과 성장** 수익으로 공정 가속, 보호, 저주, 자동판매, 특수 재료 등을 해금한다. 새 기능을 도면에 적용해 이익, 생산 속도, 재료 효율을 개선한다.
6. **반복과 엔딩** 제작과 판매를 반복해 공방을 성장시킨다. 자유 판매 중심 엔딩 조건은 미정이다.

화면	역할
코드 에디터	사각형 코드 입력창. 상하좌우 가장자리를 드래그해 크기를 조절한다. 상단에 화살표 실행 버튼을 배치한다. 숫자 위에 커서를 올리면 문맥에 맞는 뜻과 재료 비용을 보여 준다.
콘솔과 실행 기록	컴파일 오류, 실행 오류, 경고를 구분한다. 실행한 줄을 강조하고 당시 인자·온도·경도 등의 값을 표시한다. 줄 단위 실행은 확장 후보로 둔다.
3D 대장간	작업대, 화로, 모루, 냉각 도구를 통해 대장장이의 행동과 공정 순서를 보여 준다. 증기와 수치 변화 등으로 공정 효과를 전달한다.
재료 진열대·상점	재료 번호, 용도, 비용과 구매 가능한 공방 기능을 보여 준다.
검사·판매 화면	기본 가격, 경도·부품 등으로 얻은 보너스, 제작비, 순이익을 영수증에 나눠 표시한다.

3. 숫자 도면 언어

소스는 영문 변수명, `0`~`9`로 구성한 숫자, 정해진 특수문자, 공백과 줄바꿈으로 작성한다. 변수는 `Material`이나 `Blade`처럼 선언과 참조에서 같은 이름을 그대로 쓴다. 괄호 앞 숫자는 함수 번호이며, 인자로 전달한 숫자는 함수의 인자 위치에 따라 재료 번호나 온도 등의 수치로 해석한다. 숫자는 여러 자리로 조합할 수 있다. Python처럼 들여쓰기로 조건 블록을 읽는 방향이다.

표현	뜻
`7`	정수 리터럴. 함수에서 재료 번호를 받는 자리에 전달하면 해당 재료 번호로 해석한다.
`Material` / `Material = 1`	`Material` 변수 참조 / `Material` 변수에 정수 `1`을 저장.
`1(...)`	번호가 `1`인 함수 호출.
`Blade = 1(Material)`	`Material`의 값으로 재료를 작업대에 올리고 반환된 작업물 참조를 `Blade`에 저장한다. 값이 `1`이면 철을 올린다.
`+`, `>=`, `? 조건:`	산술, 비교, 조건문 예시. 조건이 참일 때 들여쓴 블록을 실행한다.
`!Blade`	`Blade`의 작업물이 판매 가능한 상태인지 검사하고 완성품으로 반환한다.

초기 재료 번호 제안

번호	재료	공방에서의 사용 예시
`0`	공기	냉각 함수에 전달해 공기로 식힌다.
`1`	철	작업대에 올려 검날의 바탕으로 쓴다.
`2`	나무	손잡이 부품이나 연료로 활용한다.
`3`	물	냉각에 쓴다. 담금질 효과는 사용 직전의 상태에 따라 달라진다.
`4`		
`5`		
`6`		
`7`	별철	검날의 열 단계가 **정확히** `7`**일 때** 냉각을 시작하면 별무늬가 새겨집니다. `6`이나 `8`에서 식히면 일반 별철 검날이 됩니다.
`8`		
`9`		

기본 공방 함수 초안

호출	공정
`1(재료 번호)`	재료를 작업대에 올리고 작업물 참조를 돌려준다.
`2(작업물, 온도)`	작업물을 목표 온도까지 가열한다.
`3(작업물)`	가열된 철을 타격해 검날의 형태를 잡는다.
`4(작업물, 냉각 재료)`	공기 또는 물을 사용해 작업물을 식힌다.
`5(작업물, 부품 재료)`	형태가 잡히고 식은 검날에 손잡이 등의 부품을 장착한다.

도면 함수·표기 레퍼런스

상태 열은 현재 프로토타입의 지원 범위와 확장 계획을 구분합니다. 현재 재료 올리기는 철, 냉각은 공기를 지원합니다. 물 냉각과 손잡이 장착은 기획 단계입니다.

함수·표기	하는 일	반환값	비용	상태	설명
`1(material)`	재료를 올려 작업물 생성	작업물 참조	철 1개 + 10G	현재 지원, 철만	재료 올리기
`2(piece, target)`	목표 열 단계로 설정	없음	호출당 2G	현재 지원	가열
`3(piece)`	가열한 작업물을 검날로 성형	없음	0G	현재 지원	타격
`4(piece, coolant)`	작업물을 냉각	없음	공기 0G	현재 지원, 공기만	냉각
`!piece`	완성품 검사 및 도면 종료	판매 가능한 완성품 등록	추가 0G	현재 지원, 함수가 아닌 문장	완성품 판정
`5(piece, material)`	손잡이 등 부품 장착	미확정	미확정	공정 기획만 존재	장착 후보
`heat(piece)`	현재 열 단계 조회	정수 0~9	0G	확장 계획	열 조회
`abs(n)`	절댓값	정수	0G	확장 계획	절댓값
`len(values)`	원소 개수	정수	0G	확장 계획	길이
`min(values)`	최솟값	정수	0G	확장 계획	최솟값
`max(values)`	최댓값	정수	0G	확장 계획	최댓값
`random(low, high)`	양 경계를 포함해 정수 추첨	정수	0G	확장 계획	난수
`set()`	빈 정수 집합 생성	집합	0G	확장 계획	빈 집합
`이름(인자...)`	`def`로 작성한 사용자 함수 호출	`return`에 따라 결정	내부 공정의 합계	확장 계획	사용자 함수

`IronBlade.num` · 기본 철 검날 도면

```
Material = 1
Blade = 1(Material)
2(Blade, 7)
3(Blade)
4(Blade, 0)
!Blade
```

`4(Blade, 0)`의 `0`을 `3`으로 바꾸면 물 냉각을 시도한다. 작업물이 식은 뒤 `!Blade` 앞에 `5(Blade, 2)`를 추가하면 나무 손잡이를 단다. 에디터에서 `4(Blade, 3)`의 마지막 `3`에 커서를 올리면 '냉각 재료 · 물'이 표시된다. `2(Blade, 7)`의 마지막 `7`에는 '목표 온도 · 7'이 표시된다.

4. 컴파일러, 가상 머신과 오류 처리

소스 → 토큰화 → 구문 트리 → 함수 인자·변수·타입 검사 → 바이트코드 → 공방 가상 머신 → 제작 상태와 완성품 판정. 조건문은 바이트코드의 분기 명령으로 처리한다. 가상 머신은 공방 함수에 도달하면 작업물 상태를 검사하고 실제 제작에서는 3D 대장장이의 작업을 실행한다. 시험 실행과 실제 제작은 동일한 공정 규칙을 사용한다.

상황	판정	플레이어 피드백
`5()` 처럼 작업물과 부품 인자를 생략함	컴파일 오류. 제작 시작 전 정지.	오류 줄과 필요한 인자 두 개를 표시.
`5(Blade, 2)` 를 호출했지만 `Blade` 를 선언한 적이 없음	컴파일 오류. 제작 시작 전 정지.	정의되지 않은 변수 `Blade` 를 표시하고, `Blade = 1(1)` 로 철을 작업대에 올려 저장하는 방법을 안내한다.
철괴를 올린 직후 손잡이를 장착함	실행 오류. 해당 공정 직전에 정지.	현재 상태가 철괴이며 형태를 잡고 식혀야 한다고 표시.
식은 검날에 물을 사용함	효과가 없는 공정. 경고 후 계속.	예시로 냉각 직전 온도 `4` , 담금질 기준 `6` , 물값 `2G` , 경도 변화 없음 표시.
`!Blade` 로 미가공 철괴를 장비로 반환함	완성품 검사 실패. 판매 단계 진입 불가.	현재 상태와 필요한 공정을 표시.

비용과 상태 처리 제안 시험 실행은 실제 재고나 비용을 차감하지 않는다. 실제 제작이 실행 오류로 멈추면 이전 공정에서 쓴 재료와 연료는 비용에 남는다. 실패한 공정의 재료는 사용 전에 검사해 차감하지 않는다. 작업대에는 마지막으로 성공한 공정까지의 작업물이 남는다. 남은 작업물의 재개·회수·폐기 방식은 결정이 필요하다.

5. 판매와 발견

아래 가격과 시간은 경제 구조를 보여 주기 위한 예시다. 첫 판매 뒤 물의 툴팁과 영수증을 비교하면 냉각 재료 하나를 바꿀 이유가 생긴다. 경도 보너스나 손잡이 보너스를 판매가에 별도 항목으로 표시한다.

완성품	제작비	판매가	개당 이익	제작 시간
공기로 식힌 철 검날	12G	28G	16G	4초
물로 담금질한 철 검날	14G	47G	33G	7초
나무 손잡이가 붙은 담금질 검	19G	69G	50G	12초

검날에 물을 쓰는 데 2G가 추가되고 판매가가 19G 오른다면 개당 이익은 17G 늘어난다. 이 예시에서는 완성검의 개당 이익이 가장 높고 담금질 검날의 시간당 이익이 가장 높다. 공정 가속으로 손잡이 장착 시간이 줄면 생산 전략도 달라질 수 있다.

플레이어는 재료의 설명에서 가능성을 읽고, 3D 공정과 검사 수치로 결과를 확인하고, 판매 영수증으로 이익을 비교한다. 온도 읽기 함수와 조건문을 도입하면 검날이 뜨거울 때만 물을 쓰도록 해 재료비를 아낄 수 있다.

6. 행운의 숫자, 상점과 해금 후보

후보	논의한 효과와 미정 사항
경도 `7` · 행운의 날	경도가 정확히 `7` 에 도달하면 숨은 특성을 발견한다. 발견 후 도감에 조건을 기록해 플레이어가 도면으로 재현하는 방향. 판매 보너스는 미정.
`777`	확정 호출은 `777(오늘의 행운 코드, result)` . 반환 작업물을 자유로운 변수명에 저장하고 `!변수명` 으로 완성한다. 추가 골드와 다이아몬드 기획은 11절을 따른다. 7 연속 사용·7배 보상은 이전 제안이며 배율은 미정이다.
`666`	저주가 깃들어 판매가 10배를 노릴 수 있으나 일정 확률로 아이템이 파괴된다. `666(Blade)` 를 공방 함수로 쓰는 안. 파괴율 25%와 보호 비용은 밸런스 예시.
`@`	저주의 아이템 파괴를 막는 보호 기능. `@666(Blade)` 표기가 후보이며 사용 비용과 정확한 문법은 미정.

후보	논의한 효과와 미정 사항
`>` · `>>` · `>>>` · `>>>>`	제작 속도 2배·3배·4배·5배에 해당하는 가속 단계 초안. 최종 표기와 적용 공정은 미정.
자동판매 · `404`	자동판매는 완성품을 판매하도록 해금하는 기능. `404`는 실패한 제작의 재료를 일부 회수하는 별도 아이디어. 두 기능의 발동 조건은 미정.
언어 확장	출력·반복·괄호·예약어 해금, `8.123` 압축 표기 등을 초기 후보로 논의했다. 기본 문법을 제공하고 상점에서 공방 기능을 해금하는 안도 있으며, 최종 해금 범위는 미정.

특수 기능은 해금한 뒤에 사용할 수 있게 한다. 문법이 맞지만 해금되지 않은 함수를 호출했을 때는 상점에서 필요한 기능을 안내하는 방식이 후보이다.

7. 게임잼 구현 우선순위

1. **자유 판매 한 바퀴** 공기·철·나무·물 번호와 가열·타격·냉각을 연결해 철 검날을 만들고 판다.
2. **숫자 컴파일러와 에디터** 변수, 정수, 숫자 함수, 반환, 공정 상태 검사, 줄별 오류 메시지를 구현한다. 초기 검증은 변수 없는 장착, 철괴에 장착, 식은 검날의 물 냉각, 정상 제작을 포함한다.
3. **발견과 수익 차이** 공기 냉각, 뜨거운 상태에서 물 냉각, 식은 뒤 물 사용, 나무 손잡이를 비교한다. 툴팁, 검사 수치, 영수증으로 차이를 전달한다.
4. **공방 성장과 자동화** 첫 수익으로 가속 또는 자동판매를 해금해 도면과 수익이 어떻게 변하는지 확인한다. 온도 읽기와 조건문으로 낭비를 줄이는 플레이를 추가한다.
5. **행운과 엔딩** 경도 `7` 발견을 먼저 시험한다. `777`, `666`, `@`의 범위와 자유 판매 중심 엔딩 조건을 정한다.

8. 보류 중인 아이디어 · 4~9번 재료와 특수 가공

상태 · 보류. 아래 내용은 이후 검토할 재료·가공 제안이다. 숫자 조건과 수치는 기획 예시다.

재료 번호 `0`~`3`은 각각 공기, 철, 나무, 물이다. `4`~`9`는 플레이어가 투입량 계산, 공정 순서, 반복, 상태 확인, 두 값의 비교, 복합 조건을 각각 활용하도록 설계한 후속 재료 후보다.

번호 · 재료	가공 규칙	플레이어가 코드로 할 일	결과
`4` 석탄	화로에 넣는 연료다. 석탄 1개당 화로의 열 단계가 `2` 오른다. 목표보다 `2` 단계 이상 높아지면 과열로 표면 품질이 떨어진다.	현재 열을 읽고 필요한 개수만 투입한다. 현재 `3`, 목표 `7`이면 `2개`를 사용한다.	가열 시간과 연료 소비를 줄여 시간당 이익을 높일 수 있다.
`5` 은	식힌 검날을 연마한 뒤 은을 입혀야 도금이 고르게 붙는다. 연마 전에 도금하면 은을 소비하고 얼룩이 남는다.	검날의 연마 상태를 확인하고 필요할 때만 연마한다.	은빛 장식으로 판매가를 높인다.
`6` 마력 수정	식은 손잡이에 수정을 끼운 뒤 약한 마력을 `3회` 주입한다. 강하게 한 번 주입하면 수정에 금이 간다.	반복문으로 약하게 주입한다. 이미 충전된 수정은 남은 횟수만 주입한다.	마력 전도 특성을 얻는다. 용뼈 손잡이에 장착하면 용의 맥동을 추가로 얻는다.
`7` 별철	검날의 열 단계가 정확히 `7`일 때 냉각을 시작하면 별무늬가 새겨진다. `6`이나 `8`에서 냉각하면 일반 별철 검날이 된다.	냉각 직전의 열을 읽고, `7`보다 낮으면 가열하고 높으면 기다린다.	별무늬로 판매가를 높인다.
`8` 서리수	검날과 서리수의 열 단계 차이가 `3` 이하이면 서리 문양을 얻는다. 차이가 크면 검날에 균열이 생긴다.	두 값의 차이를 절댓값으로 계산한다. 차이가 크면 공기	서리 문양을 얻는다. 별철의 조건도 함께 충족할 수 있다.

번호 · 재료	가공 규칙	플레이어가 코드로 할 일	결과
		로 먼저 식힌 뒤 서리수를 쓴다.	
`9` 용뼈	철 검날, 은도금 검날, 별철 검날 중 완벽하게 가공한 것 하나를 매개체로 사용한다. 용뼈와 매개체를 `100°C`의 물에서 `3초` 동안 처리한다.	검날 상태와 수조 온도를 확인하고, 정제값을 계산해 공정에 전달한다.	정제 용뼈를 손잡이로 쓸 수 있다. 마력 수정과 조합해 고가 장비를 만들 수 있다.

용뼈 정제 예시

- 완벽하게 가공한 검날의 조건은 성형과 냉각 완료, 균열 없음, 최고 마감 등급이다.
- `DragonBone = 9`로 재료 번호를 선언한다. 숫자 함수 `10`을 절댓값 함수로 배정하는 안에서는 `Refinement = 10(DragonBone + 4)`로 정제값 `13`을 계산한다. 에디터 툴팁은 `10 · 절댓값`으로 표시한다.
- `13`은 정제 공정에 전달하는 설정값이다. 공방은 검날, 수조 온도, 설정값을 검사한 뒤 정제 공정을 수행한다.
- 검날의 `열 단계`와 수조의 `온도(°C)`는 별도 값으로 표시한다.

입력 예시	공정 결과
정제값 `13`, 완벽한 검날, 수조 `100°C`	`3초`간 정제해 장착 가능한 용뼈를 얻는다.
정제값 `13`, 완벽한 검날, 수조 `95°C`	시작 전에 멈추고 `수조 온도 95°C / 필요 100°C`를 표시한다.
정제값 `12`, 완벽한 검날, 수조 `100°C`	시작 전에 멈추고 `정제값 12 / 필요 13`을 표시한다.

재료 조합을 발견하는 예시

별철 검날이 열 단계 `7`이고 서리수가 열 단계 `4`이면 두 재료의 조건을 함께 만족한다. 냉각 한 번으로 별무늬와 서리 문양을 모두 얻을 수 있다. 진열대에서 재료별 조건을 읽고, 제작 결과와 판매 영수증에서 어떤 특성이 함께 붙었는지 확인하는 흐름을 제안한다.

9. 확정 예약어와 인게임 적용 프롬프트

기록일 @2026년 11월 8일

상태 · 예약어 대응표 확정, 구현 지시 정리. 보라색 강조를 적용하며, 구체적인 색상 `#C586C0`은 초기 제안입니다. 아래 내용은 구현 요청이며 실제 구현 완료 여부는 별도로 확인해야 합니다.

이전 초안과의 관계 앞 절의 `? 조건:`과 절댓값 함수 `10(...)`은 이전 제안입니다. 이번 적용 기준은 `if → 1?`, `abs → 101()`입니다.

인게임 숫자 예약어 및 구문 강조 적용 프롬프트

Unity 6 프로젝트 `2026Unity-GameJam`의 인게임 코드 에디터와 도면 실행 시스템에 아래 예약어를 적용해주세요. 현재 프로젝트의 코드와 기획을 기준으로 에디터, 토크나이저, 파서, 실행기 구조를 확인한 뒤 구현해주세요.

예약어 대응표

기존 문법·함수	인게임 표기
`while()`	`888()`
`for()`	`123()`

기존 문법·함수	인게임 표기
`break`	`!!`
`continue`	`~~`
`if()`	`1?`
`elif()`	`01?`
`else`	`0?`
`def()`	`333()`
`return`	`<<`
`heat()`	`100()`
`abs()`	`101()`
`len()`	`1111()`
`min()`	`000()`
`max()`	`999()`
`random()`	`66()`
`set()`	`1010()`
오늘의 행운 적용	`777(오늘의 행운 코드, result)`
낮 확인	`12()`
밤 확인	`24()`
현재 시각 조회	`24(60)`

괄호, 콜론, 들여쓰기와 인자 배치는 현재 프로젝트의 문법 구조에 맞춰 적용해주세요. 반복문, 조건문, 함수 정의, 함수 호출을 각각 올바른 문법 요소로 처리해주세요.

1. 영문 표기와 숫자·기호 표기를 같은 기능으로 연결해주세요.

두 표기가 동일한 내부 명령 또는 함수 식별자를 사용하도록 구현해주세요.

예를 들어 `abs(-3)`과 `101(-3)`은 같은 절댓값 기능을 호출하며 결과가 모두 `3`이어야 합니다. `heat(Blade)`와 `100(Blade)`도 동일한 작업물의 열을 읽어야 합니다.

실행 로직을 공유하고 매개변수 검사, 반환값, 실행 순서, 비용, 상태 변화, 오류 조건도 같게 적용해주세요. 한 도면에 영문 표기와 숫자·기호 표기가 섞여 있어도 정상적으로 해석되어야 합니다.

미구현 기능은 현재 프로젝트에서 동작 정의를 확인해주세요. 정의가 부족한 부분은 필요한 결정을 구체적으로 보고해주세요.

2. 문맥을 기준으로 예약어를 인식해주세요.

숫자와 기호가 등장한 위치에 따라 의미를 구분해주세요.

- `Value = 100`의 `100`은 숫자 값입니다.
- `100(Blade)`의 `100`은 `heat` 함수입니다.
- `Value = 888`의 `888`은 숫자 값입니다.
- 반복문 위치의 `888(조건)`은 `while`입니다.
- `000(values)`의 `000`은 `min`이며, 원문 표기 `000`을 유지해야 합니다.
- 문자열과 주석 내부의 내용은 예약어로 해석하지 않아야 합니다.

- 일반 변수명에 포함된 영문 문자열을 예약어로 분리하지 않아야 합니다.

정수 변환 전에 원문과 문맥을 확인하여 `000()`과 `01?`를 인식해주세요. `01?`는 선행 0을 유지한 하나의 예약어이며, `0`과 `1?`로 분리하거나 `1?`로 정규화하지 않아야 합니다.

`!!`, `~~`, `<<`, `1?`, `01?`, `0?`는 각각 하나의 복합 토큰으로 처리해주세요. 기존 연산자와 문법의 충돌을 확인하고 토큰 우선순위를 명확히 정해주세요. 기존 완성품 처리 문법 `!Blade`와 반복문 종료 `!!`도 구분해주세요.

3. 예약어를 보라색으로 강조해주세요.

대응표에 있는 영문 예약어·내장 함수명과 숫자·기호 표기를 동일한 보라색으로 표시해주세요.

초기 색상은 `#C586C0`으로 제안합니다. 기존 테마 설정에 통합하거나 Inspector에서 조정할 수 있도록 구성해주세요.

예약어 토큰 범위만 강조해주세요. `100(Blade)`에서는 `100`을 보라색으로 표시하고, 괄호와 변수에는 기존 강조 규칙을 적용해주세요. `1?`, `01?`, `!!` 등은 토큰 전체를 같은 색으로 표시해주세요.

실행 해석과 구문 강조가 같은 토큰 정보를 활용하도록 구성해주세요. 입력 중인 미완성 코드에서도 강조가 갱신되어야 합니다.

입력·삭제·붙여넣기 과정에서 커서, 선택 영역, 스크롤, 줄바꿈, 들여쓰기, 한글 입력 조합이 정상적으로 유지되는지 확인해주세요.

TMP 리치 텍스트를 사용한다면 표시용 태그와 실제 소스를 분리해주세요. `<<`, `<`, `>`가 정확히 표시되고, 저장하거나 복사한 소스에 색상 태그가 포함되지 않아야 합니다.

4. 두 표기의 대응 관계를 툴팁으로 안내해주세요.

예약어 위에 마우스를 올리면 숫자·기호 표기, 영문 이름, 기능 설명을 함께 표시해주세요. 기존 툴팁 시스템이 있다면 활용해주세요.

표시 예시	설명
`888 ↔ while`	조건이 참인 동안 블록을 반복합니다.
`123 ↔ for`	지정된 대상이나 범위를 순회합니다.
`!! ↔ break`	가장 가까운 반복문을 종료합니다.
`~~ ↔ continue`	현재 반복의 남은 부분을 건너뛰고 다음 반복으로 이동합니다.
`1? ↔ if`	조건이 참이면 블록을 실행합니다.
`01? ↔ elif`	같은 조건문에서 앞선 조건들이 모두 거짓이고 현재 조건이 참이면 블록을 실행합니다.
`0? ↔ else`	연결된 앞 조건 분기에서 실행할 블록이 선택되지 않았을 때 실행합니다.
`333 ↔ def`	사용자 함수를 정의합니다.
`<< ↔ return`	현재 함수를 종료하고 호출한 위치로 결과를 돌려줍니다.
`100 ↔ heat`	작업물의 현재 열 단계를 읽습니다.
`101 ↔ abs`	절댓값을 구합니다.
`1111 ↔ len`	원소 개수를 구합니다.
`000 ↔ min`	최솟값을 구합니다.
`999 ↔ max`	최댓값을 구합니다.
`66 ↔ random`	지정한 범위에서 무작위 값을 구합니다.
`1010 ↔ set`	중복 원소를 허용하지 않는 집합을 만듭니다.

표시 예시	설명
`777 · 오늘의 행운 적용`	오늘의 행운 코드와 제작 결과 작업물을 받아 행운이 적용된 작업물을 반환합니다. 반환값을 원하는 변수에 저장하고 `!변수명`으로 완성합니다.
`12() · 낮 확인`	낮이면 1, 밤이면 0을 반환합니다.
`24() · 밤 확인`	밤이면 1, 낮이면 0을 반환합니다.
`24(60) · 현재 시각`	자정부터 지난 분을 0~1439 범위의 정수로 반환합니다.

인자 형식, 반환값, 난수 범위의 경계 포함 여부 등 세부 안내는 실제 구현과 일치시켜주세요.

5. 향후 코드 표기 번역에 사용할 대응 데이터를 구성해주세요.

다음 정보를 하나의 대응 데이터에서 관리해주세요.

- 내부 기능 식별자
- 영문 표기
- 숫자·기호 표기
- 문법 종류
- 설명 또는 현지화 키

파서, 구문 강조, 툴팁에서 이 정보를 함께 활용하고, 향후 영문 코드와 숫자·기호 코드 사이의 표기 변환에도 사용할 수 있도록 구성해주세요.

변환은 토큰의 의미를 기준으로 처리할 수 있어야 합니다. 함수 호출 `100(Blade)`는 `heat(Blade)`로 변환하지만, 숫자 리터럴 `Target = 100`과 문자열·주석 내용은 그대로 유지하는 방식입니다.

현재 작업 범위는 예약어 별칭, 대응 데이터, 보라색 강조, 툴팁입니다. 표기 번역 화면은 후속 기능으로 준비해주세요.

6. 실제 인게임 실행과 편집 동작을 확인해주세요.

다음 항목을 확인해주세요.

- 영문 도면과 숫자·기호 도면의 실행 결과가 동일한지
- 중첩된 반복문과 함수에서 `break`, `continue`, `return`이 정확한 범위에 작용하는지
- 숫자 리터럴, 문자열, 주석을 예약어로 오인하지 않는지
- `000()`과 `01?`를 포함한 복합 기호가 저장·불러오기 이후에도 유지되는지
- `if → elif → else`와 `1? → 01? → 0?`가 같은 분기를 선택하며, 여러 `elif` 중 처음 참인 분기만 실행되는지
- `elif`와 `01?`가 연결할 `if` 없이 사용되거나 `else` 뒤에 나타나면 문법 오류로 안내하는지
- 구문 강조와 툴팁이 실제 편집 화면에서 정상적으로 작동하는지
- 기존 제작과 완성품 처리 흐름이 정상적으로 실행되는지

완료 후 수정한 파일과 관련 함수, 사용할 수 있는 문법 예시, 인게임 적용 방법, 확인 결과를 보고해주세요. 구현 과정에서 판단한 사항과 추가 결정이 필요한 사항도 함께 알려주세요.

10. 알고리즘·자료구조를 활용하는 추가 가공 기획

정리일 @2026년 11월 9일

상태 · 선택한 아이디어 네 가지의 구체화 제안. 사용자가 선택한 방향은 겹겹이 감싸는 마법 주조, 중복 금지 룬, 시간이 지나면 강도가 감소하는 마력 타격, 미지의 공명 온도 탐색이다. 아래 수치, 보상명, 실패 처리, 해금 순서와 세부 동작은 초기 제안이며 구현 완료를 의미하지 않는다.

기본 제작과 연결

기존 재료 올리기 → 가열 → 타격 → 냉각 → 선택적 부품 장착 → 완성품 검사·자유 판매 흐름에 추가 가공을 연결한다. 기본 검부터 판매할 수 있고, 추가 가공으로 제작비와 시간을 더 투자해 높은 가치를 얻는다. 마력 타격은 기본 타격·냉각 구간을 확장하고, 나머지는 추가 가공이나 별도 부품 제작으로 연결한다.

현재 문서상 프로토타입은 철과 공기 냉각을 지원하며, 특수 재료와 확장 문법은 계획 단계다. 8절의 마력 수정 등 재료 후보는 보류 상태이므로, 아래 수정 부품은 해당 후보를 구체화한 제안으로 취급한다.

기획	제작에서의 역할	코드로 해결하는 문제	완성품 보상 제안
겹겹이 감싸는 마법 주조	검날에 보호층을 쌓고 역순으로 해제	작업 순서 기록, 스택	결계 특성
중복 금지 룬	서로 다른 계열의 룬 각인	집합, 중복 검사	복합 룬 특성
마력 타격 리듬	감소하는 임시 강도를 유지하고 냉각으로 확정	시간 확인, 조건 제어	마력 단조 등급
미지의 공명 온도 찾기	장착할 수정의 숨은 활성 온도 탐색	이진 탐색	공명 수정 부품

10.1. 겹겹이 감싸는 마법 주조

목표 봉인층을 쌓은 순서를 기록하고 역순으로 해제하여 결계 특성을 얻는다.

식힌 검날을 마법 주형에 넣고 여러 종류의 봉인층으로 감싼다. 봉인층은 바깥으로 차례대로 쌓이며, 주조가 끝나면 가장 바깥층부터 해당 방식으로 해제한다.

봉인층	해제 방법	연결할 기존 공정
열 봉인	지정한 열 단계까지 가열	`2()`
수 봉인	물을 사용해 냉각	`4()`
충격 봉인	한 번 타격	`3()`

주형 외피를 공정 대상으로 지정하는 기능을 추가한다. 검날과 외피의 상태를 구분하여 외피의 가열·냉각·타격 효과를 판정한다. 열 봉인의 정확한 목표 열 단계와 주형 안정화 조건은 미정이다.

제작 절차

1. 봉인 재료 하나를 사용하고 종류를 목록 끝에 기록한다.
2. 다음 봉인 재료도 같은 방식으로 기록하며 층을 쌓는다.
3. 주형 안정화가 끝나면 목록의 마지막 봉인을 확인한다.
4. 해당 봉인에 맞는 공정을 실행한다.
5. 해제 성공 후 마지막 기록을 꺼내고, 남은 봉인을 반복 처리한다.

열 → 충격 → 수 순서로 감쌌다면 수 → 충격 → 열 순서로 해제한다. 마지막에 넣은 값을 먼저 꺼내는 스택을 활용한다. 실제 해제가 성공한 뒤 기록을 제거하여 작업물과 목록 상태를 맞춘다.

난이도와 보상

- 초반에는 봉인 두 개를 직접 선택하며 규칙을 익힌다.
- 이후 구입한 봉인 묶음의 순서와 길이가 달라진다.

- 길이가 달라져도 작동하는 도면을 작성하는 것이 자동화 목표다.
- 잘못된 공정을 적용하면 해당 층에 균열이 생겨 그 층의 결계 보너스를 잃는 안을 제안한다.
- 손상된 층도 올바른 공정으로 제거할 수 있어 완성까지 진행할 수 있다.
- 정상 해제한 층수에 따라 보상을 결정하고, 한 번의 주조에서 보상을 받는 최대 층수를 제한한다. 구체적인 상한과 가격은 미정이다.

화면과 필요 기능

봉인층이 쌓이고 벗겨지는 모습을 3D 주형에 표시한다. 실행 기록에는 추가한 봉인, 현재 해제할 봉인, 남은 층수를 표시한다.

필요 기능은 봉인 적용·해제, 주형 외피 참조, 목록 끝에 추가·마지막 원소 확인·마지막 원소 제거, 남은 층수 조회다. 새 기능의 숫자 표기는 미정이다.

10.2. 중복 금지 룬

목표 보유 룬에서 서로 다른 계열을 골라 각인하고, 중복 때문에 자동 제작이 중단되는 일을 방지한다.

초기에는 룬 슬롯 세 개를 제안한다. 같은 계열은 하나만 각인할 수 있으며, 서로 다른 세 계열을 채우면 '삼중 공명' 특성을 얻는다. 중복 여부는 개별 룬의 고유번호가 아닌 계열로 판정하므로, 등급이 다른 불꽃 룬도 같은 계열이다.

입고 목록이 불꽃, 불꽃, 바람, 서리, 바람인 경우 다음처럼 처리한다.

확인한 룬	처리	사용한 계열 집합
불꽃	각인	불꽃
불꽃	건너뛰기	불꽃
바람	각인	불꽃, 바람
서리	각인 후 슬롯 완료	불꽃, 바람, 서리

확정 문법과 연결

1. `1010()`으로 사용한 계열의 집합을 만든다.
2. `123()`으로 보유 룬을 순회한다.
3. 이미 사용한 계열이면 `~~`로 건너뛴다.
4. 각인 성공 후 해당 계열을 집합에 추가한다.
5. `1111()`로 종류 수를 확인하고 세 개가 되면 `!!`로 종료한다.

집합 생성에 더해 원소 추가와 포함 여부 확인 기능을 지원해야 한다. 해당 조작의 표기는 미정이다.

실패·재고 처리와 성장

- 중복 각인을 시도하면 재료 사용 전에 오류를 알리고 차감하지 않는 방식을 제안한다. 기존 문서의 공정 전 상태 검사 원칙과 연결한다.
- 서로 다른 룬이 두 종류뿐이면 두 슬롯을 채운 상태로 판매할 수 있다.
- 초반에는 서로 다른 세 종류 채우기를 목표로 한다.
- 이후 같은 계열 중 높은 등급의 룬을 선택하도록 확장할 수 있다.
- 슬롯 수, 계열별 효과, 등급별 보너스, 삼중 공명 가격은 미정이다.

필요 기능은 룬 목록·계열·등급 조회, 각인, 남은 슬롯 조회, 집합 추가·포함 검사다.

10.3. 마력 타격 리듬

사용자 지정 방향 타격 후 시간이 지나면 강도가 내려가며, 일정 주기 안에 다시 타격해야 강도를 유지할 수 있다.

이를 타격으로 상승하고 시간이 지나면 감소하는 '임시 단조 강도'로 구체화한다. 냉각 시작 시의 강도를 기록하여 완성품 등급에 반영하는 방식은 제안이다. 마력 단조가 해금된 작업물에 적용한다.

항목	초기 규칙 제안
임시 단조 강도	`0~9`
타격 효과	타격 완료 시 `+3`
유지 시간	타격 완료 후 1초 동안 감소하지 않음
감소	유지 시간이 끝난 뒤 0.5초마다 `-1`
목표 구간	강도 `6~8`에서 냉각 시작
과도한 타격	타격 결과가 `9`를 넘으면 과단조 판정으로 최고 등급 상실
확정 시점	냉각 시작 시 강도를 기록하고, 냉각 성공 시 등급 확정

강도가 6이면 바로 냉각하여 목표를 달성한다. 한 번 더 타격하여 9가 되었다면 조금 기다린 뒤 냉각하거나, 다음 제작부터 타격 횟수를 줄여 도면을 개선할 수 있다.

강도가 0까지 내려가도 검날 형태는 남으며 마력 단조 보너스가 약해진다. 다시 가열하고 타격해 회복할 수 있지만 재가열 비용과 시간이 추가된다. 과단조 후 실제 강도 처리와 최고 등급 회복 가능 여부는 추가 결정이 필요하다.

재료마다 다른 제어 문제

재료 설명에 한 번 타격했을 때의 강도 상승량과 타격 후 유지 시간을 공개한다. 이 값이 재료마다 다르면 고정 횟수 반복에서 상태에 맞춘 타격·대기·냉각 제어로 발전한다.

현재 강도, 다음 감소까지 남은 시간, 공정 소요 시간을 읽을 수 있어야 한다.

시간 처리

- 도면을 실행하는 제작 시계를 기준으로 강도를 감소시킨다.
- 가열·타격·냉각·대기는 제작 시간을 소비한다.
- 조회와 계산만 반복하는 무한 실행에는 실행 한도를 둔다.
- 시험 실행에서도 같은 제작 시간과 감소 규칙을 계산하여 실제 제작과 일치시킨다.
- 타격 소요 시간, 가속 시 유지 시간과의 관계, 타격과 감소가 같은 시각에 발생할 때의 처리 순서는 구현 전에 정한다.

필요 기능은 임시 강도 조회, 유지·감소 시간 조회, 대기, 공정 소요 시간 조회, 냉각 시작 강도 기록과 등급 판정이다. 새 숫자 표기는 미정이다.

10.4. 미지의 공명 온도 찾기

목표 장착할 마력 수정의 숨은 활성 온도를 적은 횟수의 검사로 찾는다.

수정마다 열 단계 `0~9` 중 하나의 공명 온도를 정한다. 현재 열 단계는 `100()`으로 읽으며, 숨은 목표는 공명 검사로 상대적인 정보만 제공한다. 여기서 온도는 게임의 열 단계다.

검사 결과	의미
`-1`	현재 온도가 공명 온도보다 낮음
`0`	공명 성공
`1`	현재 온도가 공명 온도보다 높음

숨은 목표가 6일 때 이진 탐색 예시는 다음과 같다.

탐색 범위	검사 온도	결과	다음 범위
`0~9`	4	낮음	`5~9`
`5~9`	7	높음	`5~6`
`5~6`	5	낮음	`6~6`
`6~6`	6	성공	종료

이 범위에서는 이진 탐색으로 최대 네 번 안에 찾을 수 있다. 한 단계씩 검사하는 도면도 작동하지만 검사 시간과 조율 비용이 들므로 탐색 횟수를 줄일 가치가 생긴다.

기존 공정과 연결

- 수정을 공정 대상으로 지원하고, 가열·냉각 후 현재 온도를 확인하여 공명 검사를 진행한다.
- 온도를 낮출 때는 기존 냉각 규칙에 맞춰 충분히 식힌 뒤 다시 가열하는 방식을 사용할 수 있다.
- 기존 `4()`의 두 번째 인자는 냉각 재료다. 목표 온도를 받는 기능은 별도 설계가 필요하다.
- 공명 성공 시 수정에 활성 상태를 기록한다.
- 활성화된 수정은 식힌 뒤 검에 장착할 수 있어, 검날 제작과 부품 제작을 연결할 수 있다.

탐색의 일관성

수정 생성 시 숨은 온도를 정하고 같은 수정에는 그 값을 유지한다. 가열·냉각이나 검사 때마다 목표를 다시 추첨하지 않는다.

시험 실행은 연습용 수정으로 진행하고 실제 수정의 목표는 실행 중 검사로 찾도록 하는 안을 제안한다. 시험 실행과 실제 실행의 공정 규칙은 동일하게 유지하고 입력 수정의 목표값을 구분한다.

필요 기능은 수정 참조와 상태 조회, 공명 검사, 활성 상태 저장, 수정 장착이다. 검사 함수의 숫자 표기, 조율 비용, 검사 시간, 부품 보너스는 미정이다.

10.5. 통합 제작 흐름과 판매

순서	작업	결과
1	철을 올리고 가열·타격하여 형태를 잡는다.	기본 검날
2	임시 강도를 유지하다 적절한 시점에 냉각한다.	마력 단조 등급
3	봉인층을 쌓고 역순으로 해제한다.	결계 특성
4	별도로 수정의 공명 온도를 찾아 활성화하고 장착한다.	공명 부품
5	중복되지 않는 룬을 골라 각인한다.	복합 룬 특성
6	`!Blade`로 검사하고 자유 판매한다.	특성별 가격 영수증

판매 화면에는 기본 가격, 단조 등급, 결계, 공명 수정, 룬 조합의 가격을 별도 항목으로 표시한다. 제작비와 총 제작 시간도 함께 보여 개당 이익과 시간당 이익을 비교하게 한다.

추가 공정을 모두 수행한 장비는 개당 가치가 높지만 비용과 시간이 증가한다. 일부 공정만 자동화하는 도면도 수익성 있는 선택이 되도록 조정한다.

10.6. 필요한 기능과 구현 순서

기능 묶음	필요한 지원	연결되는 기획
작업물·재료 조회	작업물 참조, 재료 특성, 부품 상태	네 기획 공통
스택용 목록 조작	끝에 추가, 마지막 원소 확인·제거, 개수 조회	봉인 주조
집합 조작	생성, 원소 추가, 포함 여부 확인	중복 금지 룬
시간·강도 제어	강도 조회, 감소 시점, 대기, 공정 시간, 냉각 시 기록	마력 타격
공명 판정	목표값 고정, 상대 온도 검사, 활성 상태 저장	공명 수정
제작 결과 표시	특성별 보너스, 재료비, 제작 시간	자유 판매

권장 구현 순서

1. 재료·작업물 상태 조회와 목록·집합 조작을 지원한다.
2. 중복 금지 룬을 구현하여 집합과 반복문을 활용한다.
3. 봉인 주조를 구현하여 순서 기록과 스택을 활용한다.
4. 공명 온도 탐색을 구현하여 조건 분기와 탐색을 활용한다.
5. 시간 기반 마력 단조를 구현하여 제작 시계와 상태 제어를 연결한다.

기존에 확정한 숫자 예약어는 9절을 따른다. 새로 필요한 공방 기능과 자료구조 조작의 숫자 표기는 기능 범위를 결정한 뒤 배정한다.

10.7. 후속 결정 항목

- 봉인 최대 층수, 열 봉인 목표 단계, 주형 안정화 조건과 비용
- 룬 계열별 효과, 슬롯 수, 룬 등급과 조합 보너스
- 마력 타격의 실제 소요 시간, 강도 상승·감소 수치, 과단조 후 처리
- 가속·일시정지·재개 시 제작 시계와 강도 감소 규칙
- 공명 검사 비용·시간, 수정 장착 조건과 활성 상태 유지 규칙
- 목록·집합 조작 및 신규 공방 기능의 표기와 인자 형식
- 특성별 판매 보너스와 시간당 수익 밸런스

11. 오늘의 행운 함수·다이아몬드·시간 조회

정리일 @2026년 11월 9일

상태 · 행운 적용 호출 형식 확정. 고정 함수 `777(오늘의 행운 코드, result)`에 매일 달라지는 세 자리 코드를 전달한다. 반환한 작업물을 원하는 변수에 저장하고 `!변수명`으로 완성품 검사·도면 종료를 수행한다. 추가 골드와 전용 부산물 다이아몬드, 구매형 행운 코드 조회, 아침·밤·현재 시각 조회 기획을 함께 적용한다. 세부 밸런스와 미결정 항목은 아래에 구분한다.

11.1. 확정 사용 문법

오늘의 행운 코드가 `812`이고, 앞선 제작 공정의 결과 작업물이 `result`에 저장되어 있다고 가정한 사용 예시다.

```
LUCKYITEM = 777(812, result)
!LUCKYITEM
```

요소	의미
`777`	오늘의 행운 코드를 받아 작업물에 행운을 적용하는 고정 함수
첫 번째 매개변수	오늘의 유효한 세 자리 행운 코드
두 번째 매개변수	앞선 제작 공정의 결과 작업물. 예시에서는 `result`
반환값	행운이 적용된 작업물로, 완성품 검사에 전달할 수 있는 값
`LUCKYITEM`	반환값을 저장한 예시 변수명. 플레이어가 자유롭게 정한다.
`!LUCKYITEM`	해당 변수의 작업물에 완성품 검사·도면 종료를 수행한다.

`result`도 예시 변수명이다. 실제로 제작한 작업물을 담고 있는 변수를 두 번째 인자에 전달한다. `LUCKYITEM`이라는 이름에 특별한 기능이나 보상 판정을 연결하지 않는다.

변수명을 바꾼 다음 표현도 같은 방식으로 동작한다.

```
LuckySword = 777(812, result)
!LuckySword
```

개정 기록 이전의 일일 번호 직접 호출 예시 `812(Blade)`는 이번 확정 형식 `777(812, result)`로 대체한다. 매일 달라지는 값은 첫 번째 인자인 행운 코드이며, 적용 함수는 `777`로 고정한다.

11.2. 오늘의 행운 코드 생성

각 자리에 `0~9`를 사용하여 세 자리 행운 코드를 무작위로 만든다.

- 하루 동안 같은 코드를 유지한다.
- 날이 바뀌면 전날과 다른 코드를 정한다.
- 기존 함수와 같은 번호는 행운 코드에서 제외한다는 기존 기획을 유지한다.
- 기본 공방 함수, 해금 전 함수, 확정된 확장 함수와 예약 표기를 기준으로 검사한다.
- 조회하거나 도면을 다시 실행할 때 코드를 재추첨하지 않는다.
- 같은 날짜의 코드와 추첨 기준은 저장하여 불러오기 이후에도 유지한다.

현재 확정된 세 자리 숫자 표기의 예시는 `888`, `123`, `333`, `100`, `101`, `000`, `999`, `777`이다. 실제 제외 목록은 전체 등록 데이터를 사용한다. `666`, `404` 등 기존 후보도 정식으로 예약되면 제외한다.

선행 0 처리 · 미정

`012` 같은 코드를 허용할지, 코드의 데이터 타입과 표시 형식을 어떻게 둘지 정해야 한다. 허용한다면 세 자리 표시를 보존하고 생성·조회·검증에서 같은 비교 규칙을 사용한다. 기존 함수 번호와 비교할 때의 정규화 규칙도 함께 정한다.

유효 후보에서 전날 코드를 제외한 뒤 추첨하는 방식을 제안한다. 유효 후보가 없을 때의 안내·처리는 구현 시 정의한다.

11.3. 검 제작과 보상

사용자 지정 방향 오늘의 행운 코드를 사용해 검을 제작하면 추가 골드를 얻고, 다이아몬드가 제작 부산물로 나온다. 다이아몬드는 오직 이 방식의 검 제작을 통해 획득한다.

항목	기획 방향
골드	행운 적용 검 제작으로 추가 획득
다이아몬드	해당 방식으로 검을 완성했을 때 발생하는 전용 제작 부산물
일반 제작	기존 판매 골드 획득 규칙 적용
실패·미완성	검 완성 조건을 충족한 시점에 보상 판정
수량·배율	추가 골드량, 다이아몬드 수량과 확률은 미정
다이아몬드 사용처	미정

제작 흐름

1. 기존 제작 공정을 진행하고 결과 작업물을 변수에 저장한다.
2. 오늘의 행운 코드를 확보한다.
3. `777(오늘의 행운 코드, 결과 작업물)`을 호출한다.
4. 반환값을 플레이어가 정한 변수에 저장한다.
5. `!변수명`으로 완성품 검사를 수행한다.
6. 유효하게 행운이 적용된 검의 완성을 기준으로 추가 보상을 판정한다.

골드는 판매 시 추가 지급하고 다이아몬드는 제작 완료 시 지급하는 방식을 제안한다. 지급 시점은 확정이 필요하다. 기존 검날 판매 가능 범위와 구분하여 행운 보상의 '검 완성' 조건도 정한다.

구현 제안

`777`은 전달한 코드와 작업물의 유효성을 검사하고 행운 적용 여부·적용 날짜를 기록한 작업물을 반환한다. 원본 작업물을 갱신하여 반환할지, 별도 결과 작업물을 생성할지는 미정이다. 어느 방식이든 동일한 제작물의 중복 지급을 방지하도록 식별 정보를 유지한다.

같은 검에 `777`을 반복 호출하거나 완성 검사를 재실행해도 보상이 중복 지급되지 않도록 적용·지급 상태를 관리한다. 재료 차감, 완성 처리, 재화 지급의 저장 상태도 일치시킨다. 코드 불일치나 잘못된 작업물 전달에 대한 구체적 오류 처리는 확정이 필요하다.

11.4. 구매로 해금하는 행운 코드 조회 기능

상점에서 기능을 구매하면 오늘의 행운 코드를 측정하고 가져올 수 있다. 조회한 코드는 `777`의 첫 번째 인자로 전달하여 자동 제작 도면에 활용한다.

기능	역할	상태
오늘의 행운 코드 조회	현재 날짜의 유효한 세 자리 코드를 가져온다.	기능 방향 확정, 조회 함수 표기와 코드 타입 미정
구매·해금	상점에서 조회 기능의 사용 권한을 얻는다.	가격·구매 재화·해금 기간 미정
조회 결과 활용	가져온 코드를 `777(코드, result)`에 전달한다.	적용 호출 형식 확정

조회 결과를 변수에 저장할 수 있도록 한다. 날짜 또는 유효 기간 정보를 함께 연결하는 방식을 제안하며, 하루가 바뀌면 다시 조회할 수 있어야 한다.

툴팁에는 현재 행운 코드, 제작 효과와 유효 날짜를 표시하는 방식을 제안한다. `777` 호출 토큰은 기존 예약어·내장 함수와 동일한 보라색 강조 대상이다. 인자 위치의 `812`는 코드 값으로 처리하고 일반 숫자 표시 규칙을 따른다.

조회 함수 자체의 숫자 표기는 별도로 정하며 등록 후 행운 코드 추첨에서 제외한다. 조회 기능 구매 전의 코드 발견 방법과 수동 입력 가능 여부는 미정이다.

11.5. 낮·밤과 현재 시각 조회

확정 표기 낮 확인은 `12()`, 밤 확인은 `24()`, 현재 시각 조회는 `24(60)`을 사용한다.

조회 기능	표기	반환값
낮인지 확인	`12()`	낮이면 `1`, 밤이면 `0`
밤인지 확인	`24()`	밤이면 `1`, 낮이면 `0`
현재 시각 조회	`24(60)`	자정부터 지난 분을 정수 `0~1439`로 반환
오늘의 행운 코드 조회	미정	현재 날짜의 유효한 행운 코드. 코드 타입과 날짜 정보 제공 방식은 미정

`12`는 정오, `24`는 자정, `24(60)`은 하루 24시간과 한 시간 60분에서 연결한 표기다. `24`는 인자가 없으면 밤 여부를 확인하고, 인자로 `60`을 전달하면 현재 시각을 분 단위로 반환한다.

현재 시각	`24(60)` 반환값
00시 00분	`0`
12시 00분	`720`
14시 30분	`870`
23시 59분	`1439`

자정이 지나면 당일 분 값은 `0`으로 돌아온다. 날짜 자체를 포함하지 않는 반환값이므로 날짜 변경 판정에는 날짜 정보 또는 별도 갱신 처리가 필요하다.

호출과 표시

- `12()`, `24()`, `24(60)`을 함수 등록 및 도움말에 반영한다.
- `24`는 인자 유무와 값에 따라 기능을 구분한다. `60` 이외의 인자에 대한 처리는 미정이다.
- 함수 토큰 `12`, `24`는 보라색으로 강조한다. `24(60)`의 `60`은 인자 값으로 표시한다.
- 툴팁은 호출 형식에 맞춰 낮 확인, 밤 확인, 현재 시각 조회를 구분한다.
- 변수명은 자유롭게 지정한다.

시간 기준과 미정 사항

낮과 밤의 전환 시각은 미정이다. 같은 시각에 낮·밤 판정이 서로 보완되도록 경계 포함 규칙을 정한다. 시간 조회 기능의 구매·기본 제공 여부도 결정한다.

오늘이 현실 날짜인지 게임 내 날짜인지 정해야 한다. 현실 날짜라면 기준 시간대와 갱신 시각을 정하고, 게임 내 날짜라면 하루 길이와 시간 진행 규칙을 정한다.

마력 타격의 제작 시계와 하루·밤낮 시계의 관계도 결정한다. 제작 가속, 일시정지, 게임 종료 중 시간 진행이 행운 코드 갱신과 시간대 판정에 주는 영향을 일관되게 정의한다.

활용 예시 · 제안

- 낮·밤 확인 결과에 따라 도면의 실행 분기를 선택한다.

- 현재 시각을 읽어 정해진 시간 구간에 제작을 실행한다.
- 날짜 변경을 감지하면 오늘의 행운 코드를 다시 조회한다.
- 현재 시각과 사용한 행운 코드를 제작 기록에 남긴다.
- 추후 시간대별 가공 조건이 추가되면 같은 조회 기능으로 대응한다.

시간대별 제작 보너스 자체는 아직 정하지 않는다.

11.6. 날짜 변경과 저장 처리

다음 경계 조건은 구현 전에 확정한다.

- 어제의 코드를 `777`에 전달한 경우
- 코드 조회 후 `777` 호출 전에 날짜가 바뀐 경우
- `777` 적용 후 `!변수명`으로 완성하기 전에 날짜가 바뀐 경우
- 행운 적용 상태에서 저장·불러오기한 경우
- 새 기능 등록으로 저장된 행운 코드가 제외 번호에 해당하게 된 경우

권장 처리안 · 제안 `777` 호출 시점에 오늘의 유효 코드인지 검사한다. 오래된 코드라면 재조회를 안내한다. 유효하게 적용한 작업물에는 적용 정보를 보존하여, 완성 전에 날짜가 바뀌어도 그 적용을 기준으로 보상을 판정한다.

실제 재화 지급은 실제 제작 완료에 연결한다. 시험 실행에서는 예상 골드와 다이아몬드를 표시한다.

11.7. 기존 행운 기획과의 관계

`777`의 용도는 이번에 오늘의 행운 코드와 작업물을 받는 적용 함수로 확정한다. 6절의 기존 7 연속 사용·7배 보상 아이디어는 이전 제안으로 기록하며, 이번 함수의 확정 보상 배율로 해석하지 않는다.

경도 `7`, 저주 `666`, 보호 `@`, 회수 `404` 등의 후보와 행운 보상의 중첩 여부는 별도로 결정한다. 예약된 함수 번호는 일일 코드 추첨에서 제외하며 다이아몬드 획득 경로는 행운 적용 검 제작으로 한정한다.

11.8. 후속 결정 항목

- 현실 날짜와 게임 내 날짜 중 기준, 하루 갱신 시각
- 행운 코드의 데이터 타입, 선행 0 표시·비교 규칙
- `777`의 적용 가능한 작업물 상태와 검 완성 판정 조건
- 반환 작업물의 원본 갱신·복사 정책과 오류 처리
- 추가 골드 지급 시점·배율, 다이아몬드 수량·확률
- 반복 적용과 날짜 변경 중 제작 처리
- 조회 기능의 가격·구매 재화·해금 기간, 구매 전 코드 발견 방식
- 조회 함수의 숫자 표기와 날짜 정보 제공 방식
- 낮·밤 전환 시각과 경계 규칙, 시간 조회 함수의 해금 여부, `24`에 `60` 이외의 인자를 전달한 경우의 처리
- 다이아몬드 사용처와 특수 가공 보상 중첩 규칙